End-to-End Data Management Pipeline for a Recommendation System

Data Management for Machine Learning (DM4ML) | Assignment 2 | 8 November 2026

Student	Suraj Gate	**BITS Student ID**	2025ag05348
Dataset	MovieLens 1M	**Repository**	Public GitHub project

Project summary

This project implements a reproducible batch data-management pipeline for a movie recommendation system. It ingests user, movie and rating data; validates data quality; performs a per-user chronological train/test split; engineers reusable features; trains a popularity baseline and a personalized collaborative-filtering model; evaluates top-K recommendations; and orchestrates the workflow with Prefect. MLflow records model parameters, metrics and artifacts.

Tools: Python 3.11, pandas, NumPy, PyArrow, SciPy, scikit-learn, Joblib, MLflow (SQLite), Prefect, Git and GitHub.

Pipeline architecture

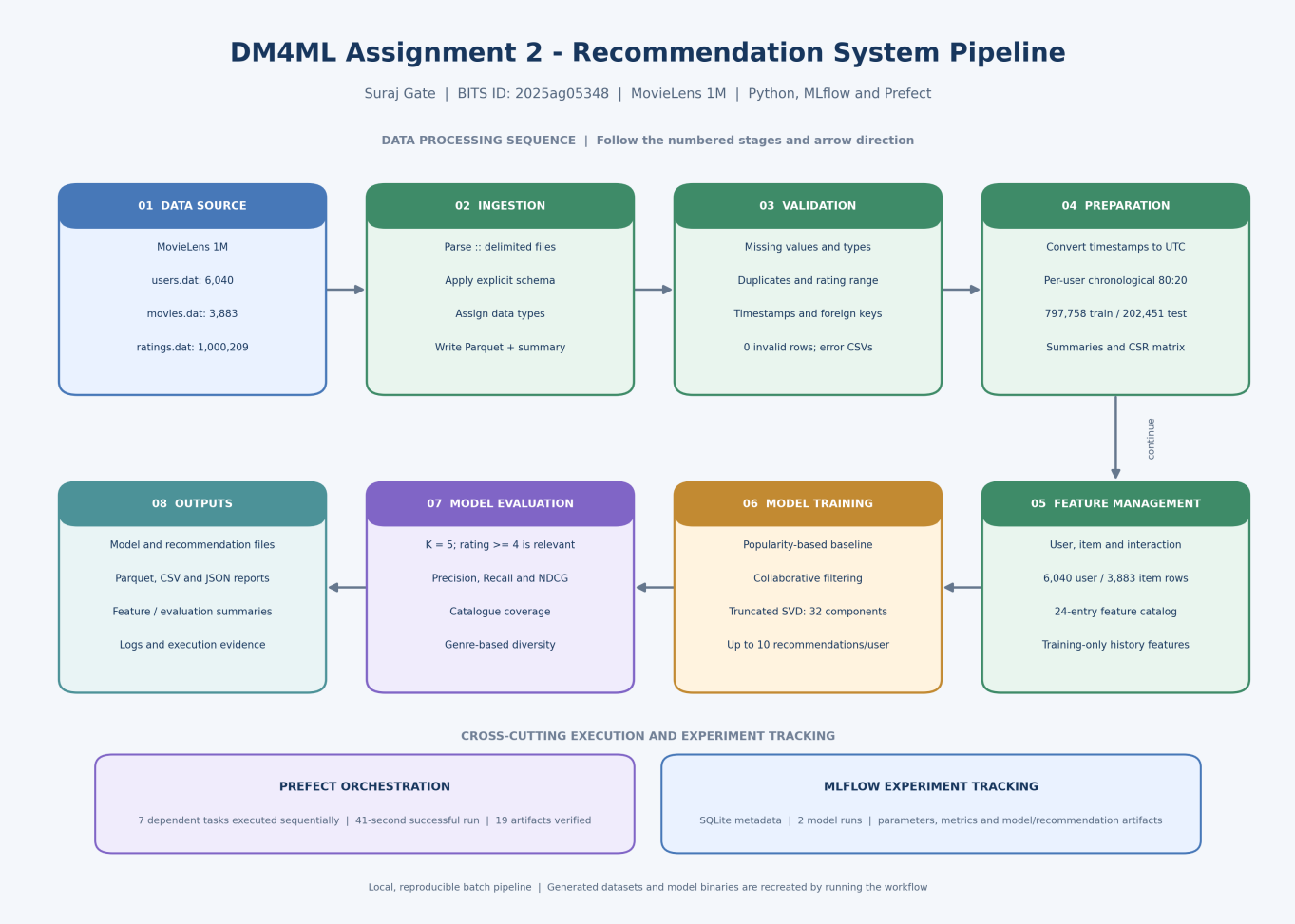


Figure 1. Implemented data flow. The upper row proceeds left-to-right; the lower row continues right-to-left. Prefect and MLflow are represented as cross-cutting services rather than data-processing stages.

The pipeline is modular and reproducible. Raw MovieLens files are downloaded locally; source modules and lightweight reports/evidence are maintained in GitHub. Generated datasets, model binaries and local service state are excluded from version control.

Data management, feature engineering and model results

Dataset and ingestion

MovieLens 1M was selected because it supplies the three required entities: user profiles, movie metadata and timestamped user-movie ratings. The source files use the **::** delimiter and are parsed into typed pandas DataFrames before Parquet storage.

Source file	Entity	Records	Fields
users.dat	Users	6,040	5
movies.dat	Movies	3,883	3
ratings.dat	Interactions	1,000,209	4

Validation and preparation

Dataset	Source rows	Valid rows	Invalid rows
Users	6,040	6,040	0
Movies	3,883	3,883	0
Ratings	1,000,209	1,000,209	0
Total	1,010,132	1,010,132	0

Automated checks covered missing required values, data types, duplicates, rating range (1-5), timestamp validity and user/movie referential integrity. All records passed; error CSVs were generated with headers and no rejected data rows. Interactions were split chronologically per user: 797,758 training rows (about 79.8%) and 202,451 test rows (about 20.2%). The split-order check passed for all 6,040 users.

Feature management

Feature output	Rows / entries	Examples / notes
User features	6,040	Activity count, average rating, positive-rating rate, preferred genre
Item features	3,883	Popularity, unique users, average rating, genre and release year
Interaction features	797,758 train; 202,451 test	History counts, historical ratings, recency, event month/day
Feature catalog	24 entries	Entity, type, definition, source and availability

Leakage controls: user/item aggregates use training data only; training history excludes the current event; test features use a training snapshot; observed ratings are labels, not predictors. Fifty-four test interactions concern movies absent from training and are retained as cold-start cases.

Model training and evaluation

Both models used 797,758 training interactions and generated up to 10 unseen-movie recommendations for each of 6,040 users (60,400 rows per model). The popularity baseline ranks by training interaction count, then average rating and MovieID. The personalized model uses scikit-learn TruncatedSVD (32 components, 7 iterations, random state 42) on the sparse 6,040 x 3,667 training matrix. MLflow tracked two runs using SQLite.

Metric (K=5)	Popularity baseline	Collaborative filtering (SVD)
Precision@5	0.08865	0.10977
Recall@5	0.02635	0.04757
NDCG@5	0.09375	0.11795
Catalogue coverage	2.19% (85 items)	16.30% (633 items)
Genre diversity	0.67655	0.71913

Evaluation protocol: ratings >= 4 in the held-out test set are relevant. Precision, recall and NDCG are macro-averaged over 5,976 users with at least one relevant test item. Coverage uses all 3,883 movies. Diversity is mean pairwise Jaccard distance between genre sets in each top-5 list. Under this evaluation setup, SVD outperformed the popularity baseline on all five reported metrics.

Orchestration, execution evidence and conclusion

The Prefect flow completed on 8 November 2026 in approximately 41 seconds. All seven tasks finished successfully: ingestion, validation, preparation, feature engineering, model training, evaluation and output verification. The final verification task confirmed the presence of 19 required pipeline artifacts.

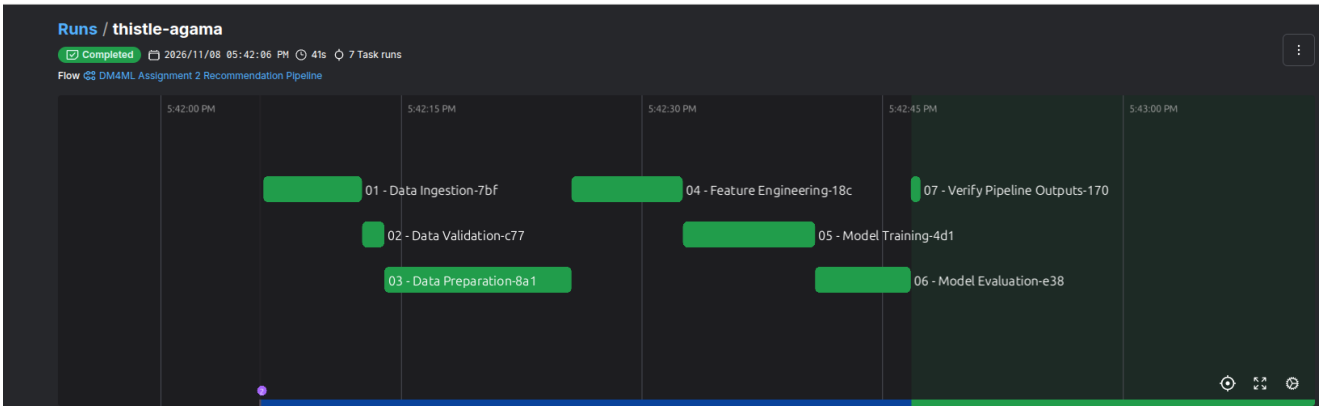
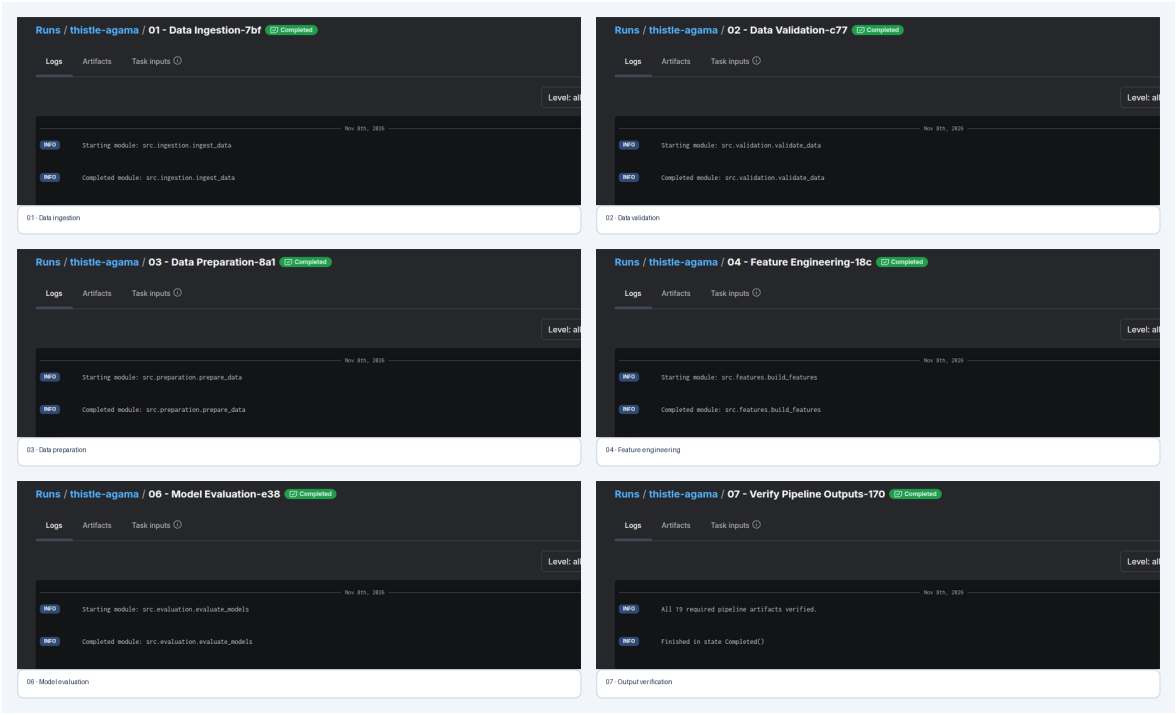


Figure 2. Prefect flow run showing Completed status, 41-second duration and all seven successful task bars.

Selected task-level execution evidence



The task-detail montage covers ingestion, validation, preparation, feature engineering, model evaluation and output verification; the overview timeline confirms Model Training also completed successfully.

Design decisions and limitations

A per-user chronological holdout approximates future recommendation. CSR storage reduces memory for sparse user-movie interactions. Popularity provides a transparent baseline, while TruncatedSVD adds personalization. Prefect enforces task order; MLflow records model runs, parameters, metrics and artifacts.

MovieLens 1M is historical and has limited profile information; test-only items are cold-start cases for SVD. This local batch workflow does not provide online serving or distributed processing. Metrics depend on the chronological split, rating >= 4 relevance threshold and K=5, and may differ under other evaluation protocols.

Conclusion

The project demonstrates a reproducible recommendation data pipeline from ingestion and quality checks through features, two models, ranking evaluation, experiment tracking and orchestration. Collaborative filtering outperformed the popularity baseline on all five reported metrics for this test split.

GitHub repository

https://github.com/2025ag05348-maker/DM4ML_Assignment_2

The repository contains source code, README setup/execution instructions, requirements.txt, architecture files, validation reports/logs, the feature catalog, evaluation results and orchestration evidence. MovieLens data and generated model/Parquet binaries are recreated locally by following the README. This PDF summarizes design and results; the README provides the detailed reproduction guide.